



南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

10 Review

CS216 Algorithm Design and Analysis (H)

Instructor: Shan Chen

chens3@sustech.edu.cn



Course Chapters

- Algorithm Analysis
- Stable Matching
- Greedy Algorithms
- Divide and Conquer
- Dynamic Programming
- Network Flow
- Computational Intractability
- Randomized Algorithms



Algorithm Analysis

- **Computational Tractability:**

- Worst-case/average-case analysis
- Efficient = worst-case polynomial-time

- **Asymptotic Order of Growth:**

- O , Ω , Θ definitions and their properties
- Common running times: logarithmic, linear, linearithmic, quadratic, cubic, polynomial, exponential, factorial, etc.

- **Five representative problems on Independent Set:**

- Interval Scheduling, Weighted Interval Scheduling, Bipartite Matching, Independent Set, Competitive Facility Location.



Stable Matching

- **One-to-One Stable Matching:**

- Example: marriage
- Gale-Sharpley algorithm
- Perfect Stable Matching
- Man optimality vs. woman optimality

- **One-to-Many Stable Matching:**

- Example: medical students applying for hospitals
- Extended Gale-Sharpley algorithm
- Stable Matching
- Student optimality vs. hospital optimality



Greedy Algorithms

- **Greedy.** Build up solution **myopically** to **optimize** some underlying criterion.
- **Scheduling:**
 - Interval scheduling: greedy algorithm stays ahead
 - Interval partitioning: “structural” bound
 - Scheduling to minimize lateness: exchange argument
 - Optimal caching: LRU, LFU for online caching and FF for offline caching
- **Graphs and Trees:**
 - Single-source/destination shortest paths: Dijkstra
 - Single-pair shortest path: A^* Search algorithm
 - Minimum Spanning Trees and k -Clustering: Prim, Kruskal, Borůvka, etc.
 - Min-Cost Arborescences: Chu-Liu’s algorithm and its $O(m \log n)$ implementation
 - Optimal prefix codes (represented as binary trees): Huffman Codes



Divide and Conquer

- **Divide and Conquer:**

- Divide up problem into several **independent** subproblems (of the same kind).
- Solve (conquer) each subproblem **recursively**.
- Combine solutions to subproblems into overall solution.

- **Applications:**

- Counting Inversions
- Closest Pair of Points
- Integer and Matrix Multiplication
- Convolution and Fast Fourier Transform (FFT)



Dynamic Programming

- **Dynamic Programming:**

- Divide up problem into several **overlapping** subproblems and combine solutions to subproblems into overall solution.
- Strategy: define subproblems, **memorize** intermediate results for later use, and **order** subproblems from “smallest” to “largest”.

- **Techniques and applications:**

- Binary choice: weighted interval scheduling
- Multiway choice: segmented least squares
- Adding a new variable: knapsack problem
- Intervals: RNA secondary structure
- DP + Divide and Conquer: Hirschberg’s algorithm for sequence alignment
- Graphs: SPFA, distance vector, negative cycle detection (and Tarjan’s trick)



Network Flow

- **Theory and algorithms:**

- **Duality:** max-flow value = min-cut capacity
- Ford-Fulkerson algorithm: improve flow value with augmenting paths
- More advanced algorithms: Capacity-Scaling, Edmonds-Karp, Dinitz.
- Adding costs to max flow: Min-Cost Max-Flow algorithms

- **Applications and extensions:**

- Bipartite Matching (and Min-Cost Max Bipartite Matching)
- Disjoint Paths
- Circulation (with supplies, demands, and lower bounds)
- Survey Design
- Airline Scheduling
- Image Segmentation



Computational Intractability

- **Basic reduction strategies:**

- Simple equivalence: $\text{INDEPENDENT-SET} \equiv_p \text{VERTEX-COVER}$
- Special case to general case: $\text{VERTEX-COVER} \leq_p \text{SET-COVER}$
- Encoding with gadgets: $3\text{-SAT} \leq_p \text{INDEPENDENT-SET}$

- **Three types of problems:**

- Decision Problems vs. Search Problems vs. Optimization Problems

- **Important complexity classes and examples:**

- **P**, **NP**, **NP**-complete, **NP**-hard
- The first **NP**-complete problem: **CIRCUIT-SAT**
- 3-SAT is **NP**-complete
- Exploiting Intractability, e.g., RSA in cryptography



Randomized Algorithms

- **Why randomize?** Can lead to simplest, fastest, or only known algorithm for a particular problem.
- **Applications:**
 - Content Resolution
 - Median and Selection
 - Global Min Cut
 - Load Balancing
 - MAX 3-SAT
- **Important math bounds for analysis.** Union bound and Chernoff bounds
- **Two types of randomized algorithms.** Monte Carlo vs. Las Vegas



Announcement of Final Exam

- **The closed-book final exam will take place on Thursday, June 11:**
 - **Time:** 4:30 pm – 6:30 pm
 - **Location:** Room 204, Zhihua Building (Lecture Hall 3)
 - Please write your answers **in English!**





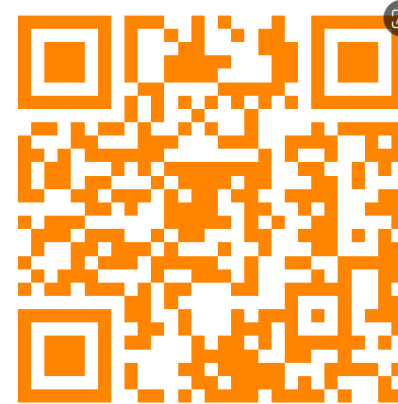
Final Note and Feedback

- **Please DO NOT publicly disclose your lab code online!**
- **请大家不要在公共平台发布自己的Lab代码，谢谢！**
- **Please provide your feedback, thanks!**



Course Evaluation

<https://tis.sustech.edu.cn>



Guidelines